# Progressive Drag Scheduling: Characterizing and Repairing Large-Rotation Editing Failures in Sparse-Controlled Gaussian Splatting

MENG GUO 48-266606
Department of Creative Informatics, Graduate School of Information Science and Technology
The University of Tokyo | Laboratory: ⟨*TODO*⟩
Own research topic: ⟨*TODO*⟩

Code: https://github.com/<TODO-user>/sc-gs-failure-analysis

## Abstract

Sparse-Controlled Gaussian Splatting (SC-GS) couples dense 3D Gaussians with a sparse set of learned control points, enabling both high-fidelity dynamic novel-view synthesis and interactive motion editing through as-rigid-as-possible (ARAP) deformation of the control graph. We present a systematic empirical study of when and why this editing pipeline fails. First, we show that the default editing mode, which initializes every ARAP solve from a linear Laplacian-editing solution, collapses for rotational drags beyond 45–60°: the rotation-blind initialization produces the classical chord-shortcut artifact, and the fixed budget of three local-global iterations cannot recover from it, yielding under-rotated, shortened limbs and torn Gaussian layouts. Second, we identify two further failure axes: an initialization-independent artifact in the rotation re-estimation stage that propagates node motion to Gaussians, and a strong sensitivity of *editability* — but not reconstruction quality — to the number of control points, including a previously unreported global "leakage" failure at high control-point counts that local rigidity metrics cannot detect. Motivated by this analysis, we propose *progressive drag scheduling*: a requested drag is subdivided into $N$ sub-steps along the target trajectory, and each sub-step's ARAP solve is warm-started from the previous solution. The method is a pure scheduling change — it requires no modification of the trained model or the solver — yet it extends the failure-onset angle from 45–60° to 90–110° ($N$=8) at under 0.8 s per drag, 16× faster than the robust-but-slow incremental reference mode. Across eight D-NeRF scenes spanning organic limbs, a mechanical hinge, an extreme kinematic chain, and a free body, progressive scheduling closes 43–80% of the rigidity gap between the original and reference modes, with monotone improvement in the schedule length on nearly every scene and angle. All results are reproducible from committed scripts, metrics, and protocol seeds.

**Keywords:** dynamic scene reconstruction; 3D Gaussian splatting; sparse control points; as-rigid-as-possible deformation; interactive shape editing; failure analysis; warm-start scheduling.

## 1 Introduction

Photorealistic reconstruction of dynamic scenes has advanced rapidly from neural radiance fields [6, 3] to point-based representations built on 3D Gaussian Splatting (3DGS) [2], which offer real-time rendering and explicit geometry. Beyond replaying a captured sequence, a natural next demand is *editing*: a user should be able to grab a reconstructed character and pose it — bend an arm, swing a tail — while the representation preserves photorealism.

SC-GS [1] is a prominent representative of this direction. It factorizes a dynamic scene into dense canonical Gaussians (appearance) and a sparse set of learned control points (motion), connected by linear blend skinning (LBS) with learned weights. Because motion lives on a few hundred control points, editing reduces to a classical geometry-processing problem: the user constrains a handful of *handle* points, and an as-rigid-as-possible (ARAP) energy [4] is minimized over the remaining control graph. The published system demonstrates compelling interactive edits and state-of-the-art benchmark quality.

Benchmarks, however, measure novel-view synthesis — not the editing behavior that motivates the sparse-control design in the first place. In this work we ask: *under what conditions does SC-GS editing actually fail, why, and how far can a minimal intervention push the failure boundary?* We deliberately study the official implementation as-is, scripting its GUI editing pipeline call-for-call so that every observation reflects the system users interact with.

Our study yields three findings. **(i)** The default editing mode (`arap_from_init`) re-initializes every ARAP solve from a *linear* Laplacian-editing solution [5]. Linear variational methods are translation-insensitive but rotation-blind [13]; under a rotational drag the initialization takes

the chord shortcut, and the solver's fixed budget of three local-global iterations cannot recover. We localize the failure onset to 45–60° of drag rotation and trace the visual symptoms — limb shortening, under-rotation, Gaussian shredding — to this mechanism. **(ii)** The alternative mode (`arap_iterative`) is robust because each mouse-move warm-starts from the previous solution, but it is two orders of magnitude slower over a large drag, is stateful (its output depends on drag history), and — surprisingly — becomes the *less* safe mode at high control-point counts, where warm-start drift accumulates into large off-region deformation ("leakage") that local rigidity metrics cannot see. **(iii)** Reconstruction quality is almost flat across a 32× sweep of control-point count, while editability degrades at both extremes — evidence that the benchmark metrics by which such systems are ranked are silent about their signature capability.

Motivated by (i) and the trade-offs in (ii), we propose **progressive drag scheduling**: subdivide a requested drag into $N$ sub-steps along the target arc and warm-start each sub-step's solve from the previous one. This imports the reference mode's robustness into the one-shot, deterministic workflow of the default mode, with a user-controllable robustness–latency dial ($N$). The method changes no trained parameters and no solver code — it is a scheduling policy over existing calls — yet it is quantitatively effective across eight scenes with qualitatively different articulation types.

**Contributions.**

1. A reproducible, GUI-faithful headless editing harness for SC-GS with geometry-level (graph edge stretch, ARAP energy, Gaussian neighbor stretch), image-level, and a novel *off-region leakage* metric (§5.1).
2. A controlled characterization of three failure axes of SC-GS editing: rotation-blind initialization (onset 45–60°), propagation-stage rotation re-estimation, and control-point-count extremes, including the leakage failure and a mode-preference inversion at high counts (§5).
3. Progressive drag scheduling, a zero-retraining repair that extends the failure onset to 90–110° at interactive latency, validated on eight scenes (§6, §7).

## 2 Summary of the Target Paper

**Target paper.** Y.-H. Huang *et al.*, *SC-GS: Sparse-Controlled Gaussian Splatting for Editable Dynamic Scenes*, CVPR 2024 [1]; official implementation at `https://github.com/yihua7/SC-GS`.

**Problem.** Given a monocular video of a dynamic scene, reconstruct it so that it can be both rendered from novel views at novel times *and* edited by a user. Prior dynamic representations optimize a dense per-point or per-voxel motion field, which reconstructs well but offers no compact handle for a person to manipulate.

**Key idea.** Decouple *appearance* from *motion*. Appearance is carried by dense canonical 3D Gaussians; motion is carried by a few hundred learned *control points*, each driven by a time-conditioned MLP. Every Gaussian follows its nearest control points through linear blend skinning with learned weights, and an as-rigid-as-possible regularizer on control-point trajectories keeps the motion field locally rigid. Because the motion field is sparse, editing becomes a classical handle-based deformation problem: the user constrains a few control points and an ARAP solve propagates the deformation to the rest, which the skinning stage then transfers to all Gaussians.

**Reported results.** On the D-NeRF synthetic benchmark SC-GS reports 43.31 dB PSNR / .997 SSIM / .0063 LPIPS averaged over eight scenes, exceeding contemporaneous dynamic NeRF and 3DGS baselines, at real-time rendering rates; qualitatively it demonstrates interactive drag-based motion editing that earlier dynamic representations do not support.

**Why we chose it.** The paper's central selling point — editability — is exactly the property its quantitative evaluation does not measure. That gap is what this report investigates.

## 3 Related Work

**Dynamic scene representations.** Dynamic NeRF variants deform a canonical field by a time-conditioned warp (D-NeRF [3]) or accelerate it with explicit grids (TiNeuVox [7], K-Planes [8]); on the splatting side, deformable 3DGS [10] and 4D Gaussian fields [9] render dynamic scenes in real time. SC-GS [1] is distinguished by *sparse motion control*, which both regularizes reconstruction and exposes an editing interface. Our work analyses that interface rather than proposing a new representation.

**Shape deformation and editing.** Handle-based deformation is classical geometry processing: linear variational methods such as Laplacian surface editing [5] are efficient but rotation-blind [13]; ARAP energies [4] restore rotation invariance by alternating local rotation fitting with global linear solves, and embedded deformation graphs [12] extend this to volumetric proxies — precisely the construction SC-GS adopts. The failure we characterize is a *system-level composition* of known ingredients: a linear initializer, a hard iteration budget, and an LBS propagation stage that re-estimates rotations from positions; to our knowledge its quantitative behavior in SC-GS had not been documented.

## 4 Understanding the Method: the SC-GS Editing Pipeline

SC-GS represents a dynamic scene as canonical 3DGS parameters plus $M$ control nodes ($M$=512 by default) with learned radii. A deformation MLP maps $(\mathbf{p}_i, t)$ to per-node translation; each Gaussian is skinned to its $K$ nearest nodes with Gaussian-kernel LBS weights, and an ARAP-style regularizer on sampled node trajectories biases training toward locally

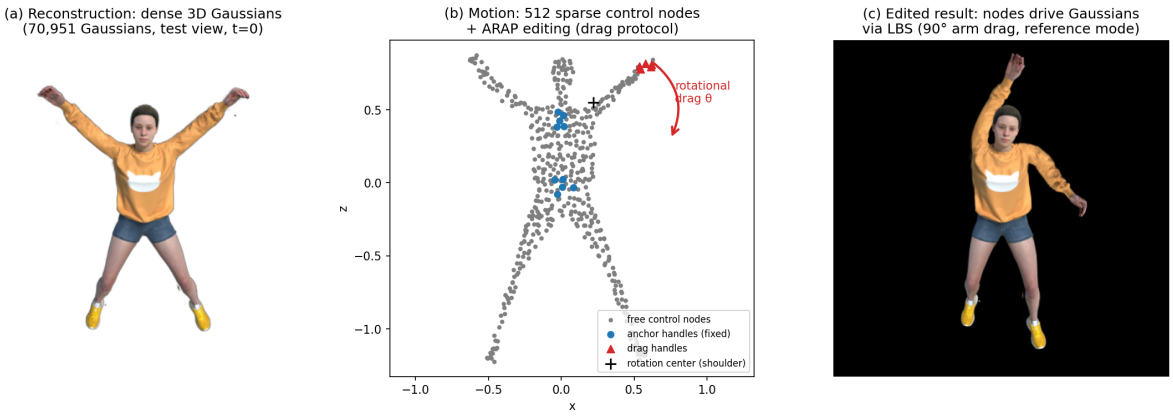


**Figure 1: SC-GS at a glance.** (a) Dense canonical Gaussians carry appearance (`jumpingjacks`, 70,951 Gaussians). (b) 512 sparse control nodes carry motion; editing constrains a drag-handle group (red, here rotated about the shoulder along an arc) and anchor groups (blue), and solves ARAP for all remaining nodes. (c) The solved node field drives the Gaussians through LBS.

rigid motion (Fig. 1). File/line references below are to the official implementation.

**Editing.** At edit time $t$, node positions $\mathbf{p}$ = nodes + $d_{xyz}(t)$ form a graph with trajectory-aware connectivity ($K$=16). The user selects a *handle* group (a clicked node plus its 2-ring) and drags it; anchors are additional handle groups with zero displacement. Writing $\mathcal{E}$ for the graph edges with weights $w_{ij}$, the solver (`utils/arap_deform.py:86`) minimizes the ARAP energy

$$E(\mathbf{p}') = \sum_{(i,j)\in\mathcal{E}} w_{ij} \left\| (\mathbf{p}'_i - \mathbf{p}'_j) - \mathbf{R}_i(\mathbf{p}_i - \mathbf{p}_j) \right\|^2, \quad (1)$$

subject to hard handle constraints, by local-global iteration — per-node SVD fitting of $\mathbf{R}_i$ alternating with a linear solve for $\mathbf{p}'$ — but runs only NUM_ITER = 3 iterations per call. Two GUI modes differ *only in initialization* (`train_gui.py:768--777`):

- `arap_from_init` (default; hereafter "**SC-GS (original)**", the baseline our method improves upon): every solve starts from the linear Laplacian-editing least-squares solution $\mathbf{p}'^{(0)} = \arg\min_{\mathbf{p}'} \|\mathbf{L}\mathbf{p}' - \mathbf{L}\mathbf{p}\|^2$ s.t. handles (`arap_deform.py:103`);
- `arap_iterative` (hereafter "**SC-GS iterative**"): every solve warm-starts from the previous drag event's solution (`init_verts`); robust but ∼100× slower over a large drag, and stateful. We use it as the robustness *reference*, not as a practical competitor.

**Propagation.** The solved node displacement field is not paired with the solver's rotations. Instead, node rotations are re-estimated from the displaced node positions by a local SVD fit (p2dR, `time_utils.py:1044`), and Gaussians are re-expressed relative to their deformed neighbor nodes with these rotations (`time_utils.py:1196--1213`). Consequently, any non-rigidity in the node field converts directly into inconsistent rotations and stretched Gaussian layouts.

# 5 An Empirical Study of Editing Failures

## 5.1 Methodology

We script the GUI pipeline headlessly and call-for-call (animation initialization → keypoint selection with $n$-ring expansion → `deform_arap` → `deform.step(node_trans_bias)` → render), so that measured behavior is exactly the interactive system's. The canonical protocol drags a limb-tip handle group (nearest node + 1-ring) along the arc of a joint rotation — *e.g.* the `jumpingjacks` hand rotated about the shoulder in the frontal plane — with the torso held by two anchor groups; intermediate limb nodes are free, so the solver must infer the limb's rotation. Handle constraints are hard and both modes satisfy them exactly (residual 0): all differences live in the free nodes. The `arap_iterative` mode applied in 1° warm-started sub-steps serves as the *reference* (it emulates GUI mouse granularity); a single `arap_from_init` solve at angle $\theta$ is exactly the state the default mode reaches at accumulated angle $\theta$, since that mode re-solves from scratch at the current absolute targets on every drag event.

Per angle $\theta \in [5°, 135°]$ we measure:

- **Graph rigidity**: relative edge-length distortion over the ARAP graph (mean/p95/max, globally and in the drag region = handle + 4 rings) and the ARAP energy of Eq. (1) between rest and deformed nodes;
- **Gaussian-level tearing**: p95 of per-Gaussian maximum neighbor-distance stretch ($K$=8 canonical neighbors) in the edited region, after LBS propagation;
- **Off-region leakage**: p95 displacement of nodes *outside* the drag region and anchors — a global metric designed to expose smooth drift that edge-based statistics cannot see (§5.4);
- **Image deviation**: PSNR/SSIM/LPIPS [11] of the rendered edit against the reference mode's render at the same $\theta$ and camera;

- **Latency**: wall-clock solve time (GPU-synced).

All protocol constants (seed points, rotation centers, axes, cameras) are serialized to JSON alongside the committed metric CSVs.

### 5.2 Failure I: rotation-blind initialization under large rotational drags

Fig. 2 quantifies the failure on `jumpingjacks`. The default mode tracks the reference up to ∼30°, then departs sharply: across 45°→60° its Gaussian-stretch statistic jumps 1.70 → 9.98 while the reference stays below 0.3 — a six-fold discontinuity that localizes the **failure onset to 45–60°**. At 135° the node edge stretch reaches 3.8× the reference (0.269 vs. 0.072) and the ARAP energy is doubled (0.093 vs. 0.048). Visually (Fig. 6, top row) the limb *under-rotates* — at 90–135° the arm remains near-horizontal instead of following the arc — *shortens* toward the chord, and the hand *shreds* into a blob of disconnected Gaussians.

The mechanism follows §4: the Laplacian initialization is a linear solve that cannot represent rotation [5, 13]; for a rotational drag it produces the chord-shortcut configuration, and three local-global iterations — each of which only re-estimates local rotations from the *current* guess — make insufficient progress from so poor a start. The reference mode avoids this entirely because its per-event increments (∼1°) keep every solve within the basin that three iterations can handle. Robustness, however, costs latency: the accumulated reference solves for a 135° drag take 12.3 s versus 0.11 s for the one-shot default — a 100× gap that explains why the fast mode is the GUI default.

### 5.3 Failure II: propagation-stage rotation re-estimation

A control experiment isolates a second, initialization-independent artifact. When the *entire* handle group is rotated about its own centroid (the GUI's rotation-drag semantics, leaving no free nodes inside the moved region), both modes produce near-identical node fields — yet **both** shred the limb beyond ∼75° (Gaussian stretch p95 ≈ 8–12). The blame therefore lies downstream of the ARAP solve: the `p2dR` stage re-estimates node rotations from displaced positions and blends Gaussians across the handle boundary with Floyd-graph LBS weights; under a large in-place rotation of a small node cluster, boundary Gaussians receive inconsistent rotation estimates and tear. This artifact is orthogonal to initialization and is *not* repaired by our method (§8); we report it as a distinct failure axis.

### 5.4 Failure III: sensitivity to control-point count

Retraining `jumpingjacks` across a 32× range of control-point counts (Tab. 1, Fig. 3) shows that **novel-view quality is essentially independent of node count** (spread 0.7 dB; the default 512 is best) — in this regime node count is an *editability and cost* knob, not a quality knob. Editing, by contrast, fails at both extremes:

**Algorithm 1** Progressive Drag Scheduling

**Require:** rest nodes $\mathbf{p}$, handle set $\mathcal{H}$, targets $H(\theta)$, steps $N$
$\mathbf{p}' \leftarrow$ NIL ▷ first step: Laplacian init
**for** $k = 1$ **to** $N$ **do**
  $H_k \leftarrow H(\theta k/N)$ ▷ slerp about drag center
  $\mathbf{p}' \leftarrow$ DeformARAP($\mathcal{H}$, $H_k$, init_verts = $\mathbf{p}'$)
**end for**
**return** node displacement field $\mathbf{p}' - \mathbf{p}$

- **Low extreme (64–128 nodes): articulation failure.** Under the standard 90° drag both solver modes produce nearly identical, heavily distorted results (region edge stretch 0.42–0.47, six times the 512-node reference); the limb cannot bend at the joint because deformation granularity is bounded by node spacing. The failure is solver-independent.
- **High extreme (2048 nodes): latency and leakage failure.** A single default-mode solve takes 3.0 s (25× the 512-node cost) and the 90° reference drag takes 200 s. More insidiously, the reference mode rips *off-region* geometry: p95 off-region node displacement reaches **1.26 scene units** (body height ≈2) — the character's legs are visibly torn sideways by an arm drag — while every *local* rigidity statistic stays moderate, because the drift is spatially smooth. This is precisely why we introduce the leakage metric. Leakage grows monotonically with node count (reference @90°: 0.09 → 0.21 → 0.38 → 0.55 → 1.26) and is ∼2× worse for the warm-started reference than for the stateless default at 2048 nodes: warm-start drift *accumulates* across the 90 sub-steps. **The preferable solver mode therefore inverts at the high extreme** — the mode that repairs Failure I is the one that fails here.

## 6 Method: Progressive Drag Scheduling

Failure I is caused by the conjunction of a rotation-blind initializer and a hard iteration budget. Rather than modifying either (retraining nothing, touching no solver code), we change *what the solver is asked to do*. Given rest handle positions $H_0$, final targets $H(\theta)$ obtained by rotating the handle group about a drag center $\mathbf{c}$ with axis $\mathbf{a}$, and a schedule length $N$:

$$H_k = H\Big(\tfrac{k}{N}\,\theta\Big), \qquad \mathbf{p}'^{(k)} = \mathrm{ARAP}_3\Big(H_k;\ \mathrm{init} = \mathbf{p}'^{(k-1)}\Big), \tag{2}$$

for $k = 1, \ldots, N$, where $\mathrm{ARAP}_3$ denotes the stock three-iteration solver and $\mathbf{p}'^{(0)}$ is the Laplacian initialization (harmless at small angles). Algorithm 1 summarizes the procedure.

The result is a deterministic function of (rest state, final targets, $N$) — unlike the reference mode it does not depend on the user's drag history, can be re-executed from a stored keyframe, and has bounded latency $N \cdot t_{\mathrm{solve}}$. Conceptually, the schedule replaces one hopeless global solve by $N$ easy

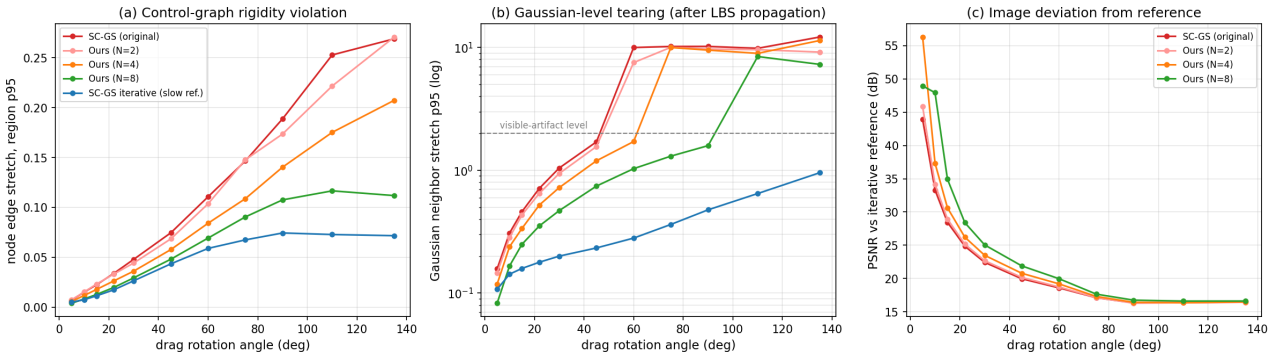


**Figure 2: Original SC-GS vs. our method under shoulder-rotation drags** (`jumpingjacks`). (a) Node-graph rigidity violation vs. drag angle. (b) Gaussian-level tearing after LBS propagation (log scale); the dashed line marks the level at which artifacts are clearly visible. (c) Image deviation from the reference. SC-GS (original) (red) departs from the slow reference (blue) between 45° and 60°; our progressive schedules (pink/orange/green, §6) recover monotonically toward the reference as $N$ grows, at a fraction of its cost.

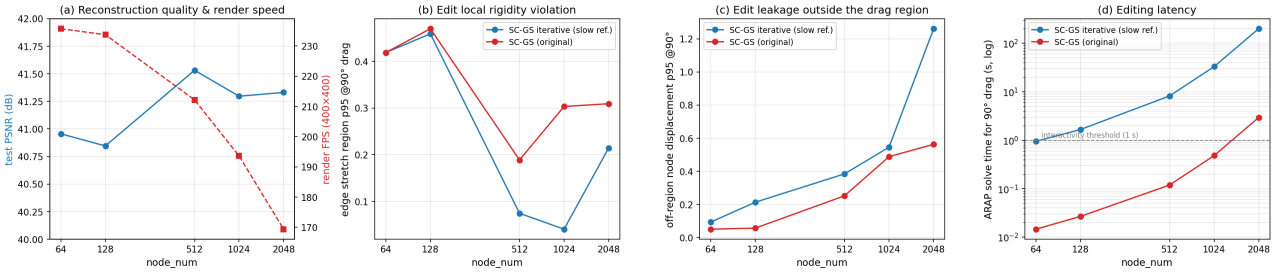


**Figure 3: Control-point count sweep on `jumpingjacks`** ($M \in \{64, 128, 512, 1024, 2048\}$, identical training). (a) Test PSNR is flat while render FPS falls 28%. (b) Edit rigidity is U-shaped. (c) Off-region leakage grows monotonically with node count and is worst for the warm-started mode. (d) Solve latency crosses the interactivity threshold.

local ones: each sub-step's increment $\theta/N$ stays within the basin in which three local-global iterations converge, and the warm start transports the solution along the arc (Fig. 4). Setting $N$=1 recovers `arap_from_init`; $N \to \theta/1°$ recovers the reference. In our implementation the schedule lives entirely in the editing driver: each sub-step reuses the exact `deform_arap(init_verts=·)` call the GUI already issues for its iterative mode, so a GUI integration is a ∼10-line loop.

# 7 Experiments

## 7.1 Execution environment and reproduction

All experiments use the official SC-GS implementation (fixed seed, 80k iterations, 400 × 400 — the D-NeRF comparison convention) on a Quadro RTX 5000; environment and build details, including a gcc-13 compilation fix, are documented in the repository. Tab. 2 verifies that our trained models reproduce the paper.

## 7.2 Main results

Robustness scales monotonically with $N$ (Tab. 3, Fig. 2): $N$=8 covers the practical editing range (≤90°) at ≤0.8 s per drag — 16× cheaper than the reference at 135° — and every image metric against the reference improves monotonically in $N$ at every angle (*e.g.* PSNR at 60°: 18.59 → 19.97 dB; LPIPS at 45°: 0.097 → 0.066). Beyond its onset, each schedule fails exactly like the default mode — the *final* sub-step's increment exceeds what three iterations absorb — implying that $N$ should scale with $\theta$; the onset table yields the simple adaptive rule $N = \lceil \theta/15° \rceil$ (fix the per-substep increment at 10–15°).

## 7.3 Cross-scene generalization

We repeat the full protocol (5 modes × 11 angles) on seven additional scenes covering qualitatively different articulation types: organic limbs (`hook` forward arm / axis $X$; `mutant` claw arm / $Y$; `standup` crouched forward arm / $X$; `hellwarrior` spread arm / $Y$), an extreme kinematic chain (`trex` tail about its root / $Z$), a *mechanical* joint (`lego` bulldozer shovel about its hinge / $X$), and an unarticulated free body (`bouncingballs`: a ball dragged along an arc above the plate / $Y$ — included to test whether the failure requires an articulated chain at all; it does not, since the arc drag alone triggers the chord-shortcut init). The ordering SC-GS (original) > Ours ($N$=2) > Ours ($N$=4) > Ours ($N$=8) > reference is **monotone at every tested angle on six of the eight scenes**, with two isolated single-point exceptions within metric noise (Tab. 4; per-scene curves for all eight scenes are in the repository). At 90° Ours ($N$=8) closes 43–

**Table 1: Control-point sweep** (`jumpingjacks`; §5.4). "ref" = `arap_iterative`, "def" = `arap_from_init`.

| node count $M$ | 64 | 128 | 512 | 1024 | 2048 |
|---|---|---|---|---|---|
| test PSNR (dB) | 40.96 | 40.85 | **41.53** | 41.30 | 41.33 |
| render FPS ($400^2$) | 235.7 | 233.7 | 212.2 | 193.6 | 169.4 |
| edge stretch p95 @90° (ref / def) | 0.42 / 0.42 | 0.46 / 0.47 | 0.07 / 0.19 | 0.04 / 0.30 | 0.21 / 0.31 |
| leakage p95 @90° (ref / def) | 0.09 / 0.05 | 0.21 / 0.06 | 0.38 / 0.25 | 0.55 / 0.49 | **1.26** / 0.56 |
| solve time @90° (ref / def) | 0.9 / 0.01 s | 1.7 / 0.03 s | 8.2 / 0.12 s | 33 / 0.48 s | 200 / 2.9 s |

(a) SC-GS (original): one-shot solve from linear init (b) Ours: progressive scheduling, N warm-started sub-steps

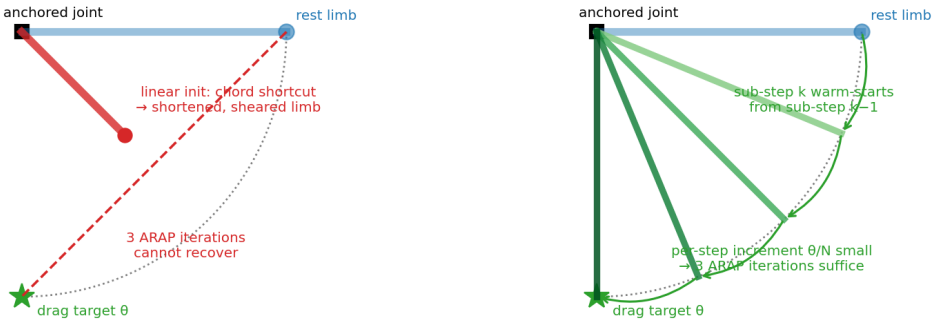


**Figure 4: Why one-shot solves fail and progressive scheduling repairs them.** (a) The linear initialization moves free nodes along the chord toward the target, shortening and shearing the limb; three ARAP iterations cannot climb back to the arc. (b) Subdividing the drag into $N$ sub-targets along the arc keeps every increment small; each solve warm-starts from the previous solution, so three iterations per sub-step suffice.

80% of the original→reference gap: 71% (`jumpingjacks`), 63% (`hook`), 80% (`mutant`), 51% (`standup`), 43% (`trex`), 79% (`hellwarrior`), 67% (`lego`), 72% (`bouncingballs`) (Fig. 5b), and image deviation from the reference improves monotonically in $N$ on all eight scenes (*e.g.* PSNR at 90°: `hellwarrior` 34.0 → 36.4 dB, `lego` 18.6 → 21.1 dB, `bouncingballs` 20.2 → 21.9 dB from SC-GS (original) to Ours ($N$=8)). Artifact *severity* varies with limb geometry — Gaussian stretch p95 at 90° is 10.2 on `jumpingjacks`' thin arm but only 1.6 on `mutant`'s thick claw — yet the qualitative conclusion is unchanged everywhere.

### 7.4 The hard case: `trex`

The `trex` tail is a single long elastic chain far from every anchor, and it stresses every mode: even the reference reaches 0.34 edge stretch at 135°. Node-level differences between modes compress (Tab. 4), and Ours ($N$=2) is statistically indistinguishable from the original — its 45–67° sub-steps already exceed the per-step failure level, so warm-starting buys nothing. Yet the *visual* verdict is unambiguous (Fig. 6, bottom row): the original mode's tail is visibly shortened and kinked, while Ours ($N$=4)/Ours ($N$=8) restore the smooth arc; image metrics agree (PSNR vs. reference at 90°: 24.7 → 25.0 → 27.7 dB from original to $N$=4 to $N$=8; LPIPS 0.039 → 0.018). The `lego` hinge is the second-hardest case for the same reason in a mechanical guise — rigid brick blocks tolerate no distortion at all, so every mode carries high absolute error, yet the ordering and the monotone image-level recovery are identical. We draw two honest conclusions: progressive scheduling helps on every scene we tested, but it is not a silver bullet for extreme kinematic chains; and geometry-level and image-level metrics can dissociate — evaluations relying on a single family of metrics would have mis-ranked these cases in both directions.

## 8 Discussion and Limitations

**A robustness–latency–statefulness triangle.** Our analysis reframes SC-GS's two editing modes as corners of a design triangle: the default mode is fast, deterministic, and restorable but rotation-blind; the reference mode is robust to rotation but slow, stateful, and — at high control-point counts — the more dangerous mode due to drift accumulation (§5.4). Progressive scheduling occupies the useful interior: bounded latency, deterministic output, and a robustness dial. Fig. 5c makes the geometry of this trade-off explicit.

**What benchmarks do not measure.** Reconstruction quality was flat across every intervention that dramatically changed

**Table 2: Reproduction on D-NeRF** (test split). Paper numbers from SC-GS Tab. 1 [1]. *Gaps beyond 0.5 dB, investigated: `trex` — best checkpoint during training reached 40.86 (≈0.2 dB is final-iteration checkpoint selection), SSIM matches and LPIPS is better than the paper's; the finest structures in the benchmark, residual consistent with densification variance. `bouncingballs` — best checkpoint 42.61, LPIPS again better than the paper's (.0142 vs .0166); the scene is dominated by large smooth motions where PSNR is highly sensitive to sub-pixel phase. †SC-GS does not report `lego` (its D-NeRF test split is known to be misaligned); we train it only as an editing test bed (mechanical joint).

| Scene | PSNR paper / ours (Δ) | SSIM paper / ours | LPIPS paper / ours | Train | Peak GPU |
|---|---|---|---|---|---|
| jumpingjacks | 41.13 / **41.53** (+0.40) | .998 / .9975 | .0067 / .0058 | 64.9 min | 1.47 GiB |
| hook | 39.87 / 39.74 (−0.13) | .997 / .9963 | .0076 / .0084 | 65.4 min | 1.91 GiB |
| mutant | 45.19 / 45.03 (−0.16) | .999 / .9990 | .0028 / .0029 | 63.5 min | 2.08 GiB |
| standup | 47.89 / 47.51 (−0.38) | .999 / .9991 | .0023 / .0025 | 65.5 min | 3.02 GiB |
| trex | 41.24 / 40.68 (−0.56)* | .998 / .9985 | .0046 / .0038 | 65.6 min | 2.07 GiB |
| hellwarrior | 42.93 / **43.00** (+0.07) | .994 / .9931 | .0155 / .0165 | 63.7 min | 1.33 GiB |
| bouncingballs | 44.91 / 42.55 (−2.36)* | .998 / .9966 | .0166 / .0142 | 68.3 min | 3.74 GiB |
| lego† | — / 25.36 | — / .9481 | — / .0386 | 63.6 min | 1.30 GiB |

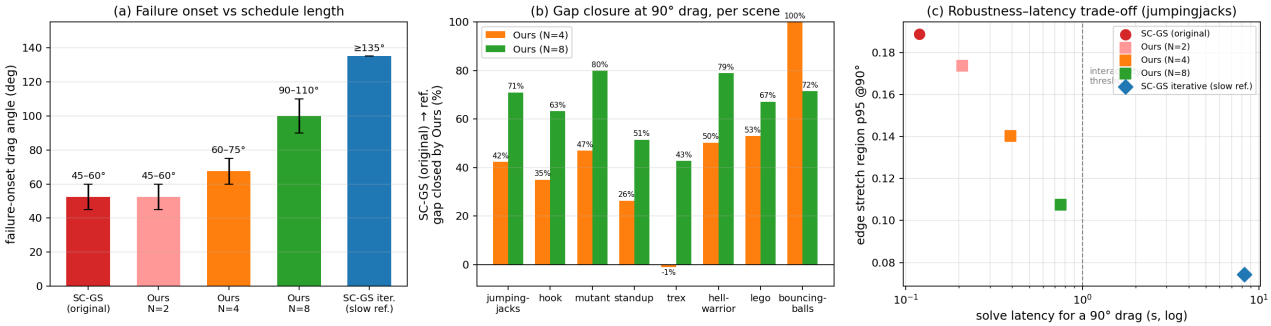


**Figure 5: How far our method moves SC-GS toward its robustness upper bound.** (a) Failure-onset angle grows with schedule length $N$: from 45–60° for SC-GS (original) to 90–110° for Ours ($N$=8) (bars: onset interval from the angle sweep). (b) Fraction of the original→reference rigidity gap closed by our method at a 90° drag, per scene (edge-stretch region p95; `trex` $N$=4 closes 0% on this node-level metric — but see §7.4 and the image-level metrics). (c) The robustness–latency plane at 90°: our schedules populate the previously empty region between the two original SC-GS modes, below the 1 s interactivity threshold.

**Table 3: Failure onset and cost** (`jumpingjacks`). Onset = first angle at which Gaussian stretch p95 exceeds the visible-artifact level (≈2).

| mode | onset | edge p95 @135° | ARAP @135° | solve @135° |
|---|---|---|---|---|
| SC-GS (original) | 45–60° | 0.269 | 0.093 | 0.11 s |
| Ours ($N$=2) | 45–60° | 0.271 | 0.091 | 0.21 s |
| Ours ($N$=4) | 60–75° | 0.207 | 0.073 | 0.39 s |
| Ours ($N$=8) | **90–110°** | 0.112 | 0.051 | 0.76 s |
| SC-GS iterative (slow ref.) | ≥135° | 0.072 | 0.048 | 12.34 s |

editability (Tab. 1). For editable representations, we argue evaluation should include editing-stress protocols of the kind proposed here — angle sweeps with rigidity, leakage, and reference-deviation metrics — rather than novel-view metrics alone.

**Limitations.** (i) Our improvement addresses the initialization failure only; the propagation-stage artifact (§5.3) and the high-count leakage failure (§5.4) are orthogonal and unrepaired — at ≥110° even $N$=8 fails, and on `trex` the reference itself is stressed. Blending the ARAP solver's own rotations instead of re-estimating them from positions, and anchoring or damping the global solve against off-region drift, are natural next steps. (ii) The iteration budget NUM_ITER = 3 was deliberately held fixed to isolate the scheduling effect; jointly scheduling sub-steps and iterations (*e.g.* extra iterations on the final sub-step) is unexplored. (iii) One drag protocol per scene, with seeds chosen once and documented; a user study or randomized-protocol sweep would strengthen external validity. (iv) The reference mode is a strong baseline, not ground truth — it drifts at extreme angles and leaks at high node counts.

**Reproducibility.** Every experiment derives from a committed artifact: one script per experiment, 29 dated entries in an experiment log, metric CSVs and figures under `results/`, and per-protocol JSON seed files (drag points, anchors, rotation centers/axes, cameras). Each figure and table here is regenerated by a single documented command (repository README). Training uses the upstream fixed seed; residual nondeterminism is limited to rasterizer atomics during densi-

**Table 4: Original SC-GS vs. our method across eight scenes: node edge stretch (region p95) at 90° / 135°.** Lower is better; the reference mode is the robustness upper bound at ∼100× the latency. Our schedules improve on the original monotonically in $N$ on every scene (two isolated exceptions: `trex` $N$=2 and `bouncingballs` $N$=8@90°, both within metric noise).

| mode | jump.jacks | hook | mutant | standup | trex | hellwarr. | lego | bounc.balls |
|---|---|---|---|---|---|---|---|---|
| SC-GS (original) | .189/.269 | .213/.318 | .237/.399 | .362/.439 | .285/.410 | .192/.307 | .420/.623 | .216/.350 |
| Ours ($N$=2) | .174/.271 | .201/.285 | .211/.337 | .344/.409 | .295/.421 | .182/.268 | .391/.580 | .198/.279 |
| Ours ($N$=4) | .140/.207 | .169/.228 | .177/.258 | .310/.389 | .285/.391 | .142/.211 | .371/.548 | .155/.202 |
| Ours ($N$=8) | .108/.112 | .134/.175 | .135/.221 | .261/.307 | .267/.371 | .113/.166 | .358/.515 | .172/.200 |
| SC-GS iterative (slow ref.) | .074/.072 | .088/.125 | .109/.157 | .165/.181 | .244/.342 | .092/.139 | .328/.402 | .155/.183 |

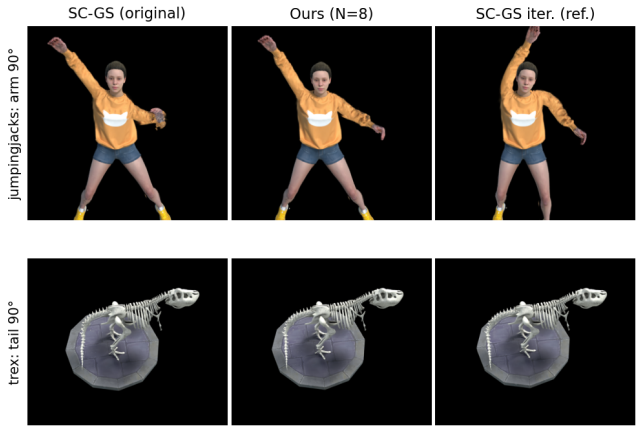


**Figure 6: Original SC-GS vs. our method at a 90° drag** on the easiest-to-see cases. Top: `jumpingjacks` — the original mode leaves the arm near-horizontal with a shredded hand; ours restores the intended lowered pose. Bottom: `trex` — the original mode shortens and kinks the tail; ours restores the smooth arc. Per-angle grids for all eight scenes are in the repository.

fication.

**Use of generative AI.** Claude Code (Anthropic Fable 5) was used as a research-engineering assistant for environment setup, code reading, experiment scripting and execution, figure generation, and draft text; the running log of tools, purposes, and human-revised parts is `docs/AI_USAGE_LOG.md` in the repository. All quantitative claims are machine-generated from the committed measurement CSVs rather than written by hand, and all protocol choices are serialized alongside the results. The author reviewed and revised the content and is responsible for it.

## 9 Conclusion

We presented a systematic failure analysis of the SC-GS editing pipeline, localizing a sharp large-rotation failure to its rotation-blind linear initialization under a fixed iteration budget, identifying an independent propagation-stage artifact, and characterizing a bidirectional failure of editability — including a previously unmeasured global leakage mode — across control-point counts at which reconstruction quality is indifferent. A minimal, zero-retraining repair — progressive drag scheduling — extends the usable drag range by roughly a factor of two at interactive latency and generalizes across eight scenes with organic, mechanical, and unarticulated geometry. The broader lesson is that editable neural representations deserve editing-centric evaluation: the capabilities that motivate their design are exactly the ones current benchmarks do not measure.

## References


[1] Y.-H. Huang, Y.-T. Sun, Z. Yang, X. Lyu, Y.-P. Cao, and X. Qi. SC-GS: Sparse-controlled Gaussian splatting for editable dynamic scenes. In *CVPR*, 2024.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph. (SIGGRAPH)*, 42(4), 2023.

[3] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *CVPR*, 2021.

[4] O. Sorkine and M. Alexa. As-rigid-as-possible surface modeling. In *Symposium on Geometry Processing (SGP)*, 2007.

[5] O. Sorkine, D. Cohen-Or, Y. Lipman, M. Alexa, C. Rössl, and H.-P. Seidel. Laplacian surface editing. In *Symposium on Geometry Processing (SGP)*, 2004.

[6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.

[7] J. Fang, T. Yi, X. Wang, L. Xie, X. Zhang, W. Liu, M. Nießner, and Q. Tian. Fast dynamic radiance fields with time-aware neural voxels. In *SIGGRAPH Asia*, 2022.

[8] S. Fridovich-Keil, G. Meanti, F. Warburg, B. Recht, and A. Kanazawa. K-Planes: Explicit radiance fields in space, time, and appearance. In *CVPR*, 2023.

[9] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang. 4D Gaussian splatting for real-time dynamic scene rendering. In *CVPR*, 2024.

[10] Z. Yang, X. Gao, W. Zhou, S. Jiao, Y. Zhang, and X. Jin. Deformable 3D Gaussians for high-fidelity monocular dynamic scene reconstruction. In *CVPR*, 2024.

[11] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

[12] R. W. Sumner, J. Schmid, and M. Pauly. Embedded deformation for shape manipulation. *ACM Trans. Graph. (SIGGRAPH)*, 26(3), 2007.

[13] M. Botsch and O. Sorkine. On linear variational surface deformation methods. *IEEE Trans. Vis. Comput. Graph.*, 14(1):213–230, 2008.